第七章　异常处理

第七章 异常处理

1. 异常处理的概念
2. c++异常处理
3. c++标准库中的异常类

1. 异常处理的概念

大型软件开发中的问题：

✓两个最大问题：

最大问题—错误连篇，不稳定的代码。

最大开销—测试、查找和修改错误。

✓两种错误：

编译错误—语法错误

运行中的错误— 1、不可预料的逻辑错误。
2、可预料的运行异常。

✓C++的异常处理仅处理运行中的异常

1. 异常处理的概念　逻辑错误与运行异常

✓**逻辑错误**：是由于不当的设计造成的，如某个排序算法不合适，导致在边界条件下，不能正常完成的排序任务。一般只有当用户做了某些出乎意料的事才会出现逻辑错误，这些错误安静地潜伏着，连许多大型的优秀软件都不能避免。就像大战之后残留的地雷，在“一切正常”中，突然进入了误区，程序发生了“爆炸”。一旦发现逻辑错误，专门为其写一段处理错误代码，就可以避免错误的发生。

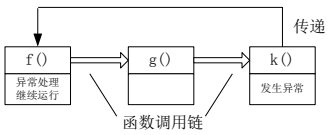
✓**运行异常**：可以预料，但不能避免，它是由系统运行环境造成的。如内存空间不足，硬盘上文件被挪离,程序中发生除0错误,打印机未打开。这些错误可以预料，通常加入一些预防代码便可以防止这些异常。

1. 异常处理的概念　异常处理的基本思想

✓**异常**：程序在运行过程中出现的错误统称为异常。C++中，异常是指从发生问题的代码区域传递到处理问题的代码区的一个对象。

✓对异常的处理称为**异常处理**。

✓C++处理异常的目的，是在异常发生时，尽可能地减少破坏，妥善地善后，而不去影响其它部分程序的运行



✓发生异常的地方在函数k()中，处理异常的地方在其上层函数f()中，处理异常后，函数k()和g()都退栈，然后程序在函数f()中继续运行。

1. 异常处理的概念　两种处理方法

✓一个强健的应用程序总是预防并处理运行中可能出现的各种异常现象，如系统I/O错误、内存溢出、数据库错误、网络错误、用户数据错误等。

✓**处理异常有两种方式：**

第一种是采用判断或分支语句实现异常处理。

第二种方法就是在异常出现时直接抛出异常，在调用函数中捕获异常，捕获异常后即进行异常处理。

```
#include<iostream.h>
int Div(int x,int y);
int main()
{ cout <<"7/3="<<Div(7,3)<<endl;
  cout <<"5/0="<<Div(5,0)<<endl;
  return 0;
}

int Div(int x,int y)
{if (y==0)
  { cout<<"除数为0，错误!!"<<endl;
    exit (0);
  }
 return x/y;}
```

用条件语句处理异常

1. 异常处理的概念

两种处理方法

✓一个强健的应用程序总是预防并处理运行中可能出现的各种异常现象，如系统I/O错误、内存溢出、数据库错误、网络错误、用户数据错误等。

✓**处理异常有两种方式：**

第一种是采用判断或分支语句实现异常处理。

第二种方法就是在异常出现时直接抛出异常，在调用函数中捕获异常，捕获异常后即进行异常处理。

```
#include<iostream.h>
int Div(int x,int y);
int main()
{ try    //检查异常
   { cout <<"7/3="<<Div(7,3)<<endl;
     cout <<"5/0="<<Div(5,0)<<endl;}
  catch(int) //捕获处理异常
   {cout<<"除数为0, 错误!!"<<endl;}
  cout<<"END"<<endl;
  return 0;
}

int Div(int x,int y)
{ if (y==0)
    throw y; //抛出异常
  return x/y;}
```

检查——抛出——捕获

第七章 异常处理

1. 异常处理的概念
2. c++异常处理
3. c++标准库中的异常类

2. c++异常处理

C++语言异常处理基本思想

- ✓ C++语言异常处理机制的基本思想是将**异常的检测与处理分离**。当在一个函数体中检测到异常条件存在，但无法确定相应的处理方法时，将**引发一个异常**，并由函数的**直接或间接调用**检测并处理这个异常。
- ✓ C++定义了三个关键字完成异常处理：
  try、throw、catch
  当try程序段检测到异常时，在try程序段中通过throw抛出异常，被抛出的异常被后面的catch语句捕获，并由catch程序段处理。
- ✓ 使用步骤：
  1、定义异常(try语句块)
  2、定义处理异常(catch语句块)
  3、抛掷异常(throw语句)

2. c++异常处理

使用try/catch捕获异常

✓在C++中，提供了语句try/catch来捕获异常，其中，try和catch分别用于定义异常和定义异常处理。定义异常是将可能产生错误的语句放在try语句块中。定义异常处理是捕获到异常后，说明如何处理异常。

```
try
{
        string str = null;  //定义字符串对象
        ProcessString(str);     //执行某个函数
}

catch (Exception e)    //定义对异常的处理
{
        cout<<"Process is error";
        exit(1); //异常退出程序
}
```


2. c++异常处理

使用throw抛出异常

➢抛出异常即检测到异常后将异常传递出去，在C++中，其采用throw语句来实现：

throw 表达式；

➢如果在try语句块的程序段中（包括在其中调用的函数）发现了异常，且抛掷了该异常，则这个异常就可以被try语句块后的某个catch语句所捕获并处理，捕获和处理的条件是**被抛出的异常的类型与catch语句的异常类型相匹配**。由于C++使用数据类型来区分不同的异常，因此在判断异常时，throw语句中的表达式的值就没有实际意义，而表达式的类型就特别重要。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }

  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

（1）如果预料某段程序代码（或对某个函数的调用）有可能发生异常，就将它放在try子句之后。如果这段代码（或被调函数）运行时真的遇到异常情况，其中的throw表达式就会抛出这个异常。

在出现异常的情况下，try语句块提示编译器到哪里查找catch块，没有紧跟try块的catch块是没有作用的。当没有发生异常的时候，几乎没有和try块相关的运行时成本。查找匹配捕获处理异常的过程只在发生异常的情况下才会进行。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }

  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```

2. c++异常处理

try-throw-catch用法解析

(2) catch子句后的复合语句是异常处理程序，捕获由throw表达式抛出的异常并处理。异常类型说明部分指明该子句处理的异常的类型，它与函数的形参是相似的，可以是某个类型的值，也可以是引用或常数。 如果某个catch语句的**参数类型与引发异常的信息数据类型**相匹配，则执行该catch语句的异常处理（捕获异常），此时，由throw语句抛出的异常信息（值）传递给catch语句中的参数。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

(3) try语句块必须出现在前，catch紧跟在后。catch之后的圆括号中必须含有数据类型，捕获是利用数据类型匹配实现的。在try{•••} 和catch(•••){ •••}语句之间不得插入任何其它C++语句。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

(4) 如果程序内有多个异常处理模块，则当异常发生时，系统自动查找与该异常类型相匹配的catch模块，查找次序为catch出现的次序。catch处理程序的出现顺序很重要，因为异常处理程序是按照它出现的顺序被检查的。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

(5) 引发异常的throw语句必须在try语句块内，或是由try语句块中直接或间接调用的函数体执行。throw语句的一般形式为:

throw exception;

这里的exception表示一个异常值，它可以是任意类型的变量、对象或常量。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

(6) catch语句的类型匹配过程中不作任何类型转换，例如unsigned int类型的异常值不能被int类型的catch参数捕获。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

try-throw-catch用法解析

(7) 如果异常错误类型为C++的类，并且该类有基类，则应该将派生类的错误处理程序放在前面，基类的错误处理程序放在后面。

```
#include<iostream.h>
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
    { cin>>i>>ch;
      if(i==0) throw 0;
      if(ch=='!') throw '!';
    }
  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}
```

2. c++异常处理

try-throw-catch用法解析

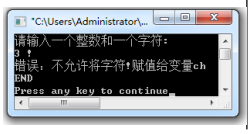
(8) 如果一个异常发生后，系统找不到一个与该错误类型相匹配的异常错误处理模块，则函数terminate()将被自动调用进行默认处理，默认功能是调用abort()终止程序的执行。错误处理函数是由set_terminate()函数来指定的，当然，程序员可以用set_terminate()函数来指定自己希望使用的错误处理函数

```
#include<iostream>
using namespace std;
void aa();
int main()
{ int i;char ch;
  set_terminate(aa);
  cout<<"请输入一个整数和一个字符:\n";
  try
   { cin>>i>>ch;
     if(i==0) throw 0;
     if(ch=='!') throw 'r';
     if(i<0) throw "error";
   }

  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char)
   {cout<<"错误：输入了字符!"<<endl;}

  cout<<"END"<<endl;
  return 0;
}

void aa()
{ cout<<"n必须为正正整数"<<endl;
  exit(-1);}
```


2. c++异常处理

try-throw-catch用法解析

(9) 如果将捕获语句写为catch(…)，可以捕获所有数据类型的异常，但**无法接受异常信息**。可以将它置于多个并列catch()语句的最后。完成类似于switch()-case语句中的default关键字的默认处理。




```
#include<iostream>
using namespace std;
int main()
{ int i;char ch;
  cout<<"请输入一个整数和一个字符:\n";
  try
   { cin>>i>>ch;
     if(i==0) throw 0;
     if(ch=='!') throw '!';
     if(i<0) throw "error";  }

  catch(int)
   {cout<<"错误：输入为0"<<endl;}

  catch(char cha)
   {cout<<"错误：不允许将字符"<<cha
        <<"赋值给变量ch"<<endl;}

  catch(...)
   { cout<<"n必须为正正整数"<<endl; }

  cout<<"END"<<endl;
  return 0;
}
```


2. c++异常处理

重新抛出异常

当catch语句捕获一个异常后，可能不能完全处理异常，在完成某些操作后，catch子句可能决定该异常必须由函数链中更上级的函数来处理，这时catch子句可以重新抛出该异常，把异常传递给函数调用链中更上级的另一个catch子句，由它进行进一步处理。重新抛出异常的表达式仍然为：

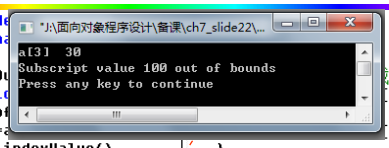
throw;

被重新抛出的异常就是原来的异常对象。

```
#include<iostream>
using namespace std;

void h()  //最低层
{throw 0;}

void g()  //中间层
{ try
   {h();}
  catch(int)
   { cout<<"catch in g\n";
     throw;// 重新抛出异常
   }
}

int main()
{ try
   {g();}
  catch(int)
   {cout<<"catch in main\n";}
  return 0;
}
```


2. c++异常处理

异常处理实例：




```
#include
using na

class Ou
{ public
   Outof
    {i=a
   int indexValue()
    {return i;}
  private:int i;
};

class Array
{ public:
  int &operator[](int i)
   {if (i<0||i>=10)
     throw OutofBounds(i);
    return a[i]; }
  private: int a[10];
};
```

```
运算符函数
[3]<<endl;

[100]<<endl;
  }
 catch(OutofBounds error)
  {cout<<"Subscript value "
  <<error.indexValue()
  <<" out of bounds\n";}
   return;
}
```

构造一个包含10个元素的数组，重载运算符[]，给数组元素赋值，进行越界检查，如果下标为负或大于10，报告越界错误。

第七章 异常处理

1. 异常处理的概念
2. c++异常处理
3. c++标准库中的异常类

3. c++标准库中的异常类

✓C++标准库提供了一个**异常类层次**结构，用来报告C++标准库中的函数执行期间遇到的程序异常情况。这些异常类可以被用在用户编写的程序中，或被进一步派生来描述程序中的异常。

✓C++标准库中的异常层次的根类为exception，定义在库的头文件exception中，它是c++标准库函数抛出的所有异常类的基类。

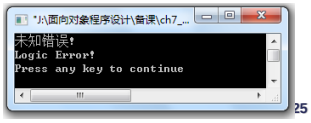
✓exception类中的虚函数what()返回一个c风格的字符串，该字符串为抛出异常提供文本描述。

✓标准C++库异常类可以分为**运行时异常和逻辑异常**。

2. c++标准库中的异常类

例：

```
#include<iostream>
#include<exception>
using namespace std;

void main()
{ try
   {exception MyError("未知错误!"); //声明一个exception类的对象
    throw (MyError);}
  catch(const exception &ee)
          {cout<<ee.what()<<endl;}
 try
  { logic_error Mylogicerror("Logic Error!"); //声明一个logic_error类的对象
    throw (Mylogicerror);}
 catch(const logic_error &le)
    {cout<<le.what()<<endl;}
}
```



25

End